API UEH Student

Base URL: `https://api.ngoisaoviet.info`

Chỉ dùng HTTPS. Request và response là JSON (`Content-Type: application/json`).

API	Method	Đường dẫn	Cần token
Đăng nhập	`POST`	`/api/login`	Không
Làm mới token	`POST`	`/api/refresh-token`	Không
Thông tin sinh viên	`GET`	`/api/student-info`	Có
Điểm danh	`POST`	`/api/attendance`	Có
Menu drawer	`GET`	`/api/menu`	Có
Tin nổi bật	`GET`	`/api/news/featured`	Không
Tin theo mục	`GET`	`/api/news?cat_uid=`	Không
Chi tiết tin	`GET`	`/api/news/{uid}`	Không

Bắt đầu nhanh

```
# 1. Đăng nhập, lấy token
curl -X POST https://api.ngoisaoviet.info/api/login \
  -H "Content-Type: application/json" \
  -d '{"mssv":"31240000000","mat_khau":"********"}'

# 2. Gọi API cần đăng nhập
curl https://api.ngoisaoviet.info/api/student-info \
  -H "Authorization: Bearer <token>"
```

Quy tắc token

- Gửi `token` trong header `Authorization: Bearer <token>` cho mọi API có cột "Cần token" là **Có**. API tin tức không cần token.
- `token` sống **60 phút**. Gặp `401` thì gọi làm mới token, rồi gọi lại API.
- `refresh_token` sống **7 ngày**. Hết hạn hoặc bị từ chối thì đăng nhập lại.
- Danh tính lấy từ token. Sinh viên chỉ thao tác được trên dữ liệu của chính mình.

---

1. Đăng nhập

`POST /api/login`

Trường	Kiểu	Bắt buộc
`mssv`	string	Có
`mat_khau`	string	Có

`200`

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "refresh_token": "Wm9vbWVy...==",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

`expires_in` tính bằng giây.

Mã	Khi nào
`401`	Sai MSSV hoặc mật khẩu: `{ "message": "MSSV hoặc mật khẩu không đúng." }`
`400`	Thiếu trường bắt buộc (xem lỗi 400)

2. Làm mới token

`POST /api/refresh-token`

Trường	Kiểu	Bắt buộc
`refresh_token`	string	Có

`200`: giống đăng nhập, trả cặp `token` và `refresh_token` **mới**.

Mã	Khi nào
`401`	Refresh token sai, hết hạn hoặc đã bị thu hồi: `{ "message": "Refresh token không hợp lệ hoặc đã hết hạn." }`
`400`	Thiếu `refresh_token`

> **Mỗi refresh token chỉ dùng một lần.**
>
> - Sau khi refresh, phải lưu ngay cặp token mới và bỏ cặp cũ.
> - Không gửi hai request refresh song song với cùng một token.
> - Dùng lại token đã thu hồi sẽ thu hồi **toàn bộ** refresh token của sinh viên đó, buộc đăng nhập lại.
> - Admin đổi mật khẩu sinh viên cũng thu hồi mọi refresh token cũ.

3. Thông tin sinh viên

`GET /api/student-info`

`200`

```
{
  "uid": 1,
  "mssv": "31240000000",
  "hoVaTen": "Nguyễn Văn A",
  "ngaySinh": "2006-10-12T00:00:00",
  "gioiTinh": true,
  "nienKhoa": "DHCQK50",
  "noiSinh": "TP. Hồ Chí Minh",
  "heDaoTao": "ĐHCQ",
  "nienHoc": "2024 - 2027",
  "hinhDaiDien": "https://randomuser.me/api/portraits/men/42.jpg"
}
```

`gioiTinh`: `true` là Nam, `false` là Nữ. Response không có mật khẩu.

Mã	Khi nào
`401`	Thiếu token, token sai hoặc hết hạn (body rỗng)
`404`	Sinh viên không còn tồn tại: `{ "message": "Không tìm thấy sinh viên." }`

4. Điểm danh

`POST /api/attendance`

Admin tạo đợt trên web quản trị và nhận **mã đợt 6 ký tự** (chữ hoa và số, ví dụ `A7K2P9`). **Mã QR chứa đúng chuỗi này.** App quét QR, lấy chuỗi làm `code` rồi gọi API. Sinh viên cũng có thể nhập tay.

Trường	Kiểu	Bắt buộc	Ghi chú
`code`	string	Có	Không phân biệt hoa thường, tự bỏ khoảng trắng hai đầu
`latitude`	number	Khi đợt yêu cầu vị trí	Từ -90 đến 90
`longitude`	number	Khi đợt yêu cầu vị trí	Từ -180 đến 180

```
{ "code": "A7K2P9", "latitude": 10.7610401, "longitude": 106.6684314 }
```

`200`

```
{
  "message": "Điểm danh thành công.",
  "checkInNumber": 1,
  "sessionName": "Điểm danh buổi 1",
  "mssv": "31240000000",
  "hoVaTen": "Nguyễn Văn A",
  "checkedInAt": "2026-09-30T09:15:32.1234567",
```

```
  "distanceMeters": 12.4
}
```

- `checkInNumber`: đây là lần điểm danh thứ mấy của sinh viên trong đợt. Luôn là `1` với đợt chỉ cho điểm danh một lần; với đợt cho phép nhiều lần thì tăng dần `1, 2, 3, ...`.
- `checkedInAt`: giờ do **server** ghi nhận, theo giờ Việt Nam (UTC+7).
- `distanceMeters`: khoảng cách tới địa điểm (mét), `null` nếu đợt không yêu cầu vị trí.

Lỗi (body dạng `{ "message": "..." }`)

Mã	Message	Nguyên nhân
`404`	Mã điểm danh không tồn tại.	Sai mã
`409`	**Bạn đã điểm danh bằng mã này rồi.**	Đợt chỉ cho điểm danh một lần và sinh viên đã điểm danh. Không xảy ra với đợt cho phép nhiều lần
`400`	Đợt điểm danh chưa bắt đầu.	Chưa đến giờ
`400`	Đợt điểm danh đã kết thúc.	Quá giờ
`400`	Đợt điểm danh này yêu cầu vị trí (latitude, longitude).	Thiếu tọa độ
`400`	Tọa độ không hợp lệ.	Ngoài phạm vi
`403`	Bạn đang ở ngoài phạm vi điểm danh (cách X m, cho phép Y m).	Quá xa địa điểm
`401`	(body rỗng)	Thiếu token, token sai hoặc hết hạn

Địa điểm: cố định tại **Cơ sở B (B1-B2)**, 279 Nguyễn Tri Phương, P. Diên Hồng, TP. Hồ Chí Minh, tọa độ `10.7610401, 106.6684314`. Bán kính cho phép do admin chọn cho từng đợt: 50, 100, 200, 300, 500 hoặc 1000 m.

Một lần hay nhiều lần: admin chọn khi tạo đợt. Mặc định mỗi sinh viên chỉ điểm danh **một lần**, lần gửi thứ hai nhận `409`. Nếu admin bật "Cho phép điểm danh nhiều lần" thì mỗi lần gửi hợp lệ (đúng giờ, đúng vị trí) đều được ghi nhận riêng và không bao giờ nhận `409`. App không biết trước đợt thuộc loại nào, chỉ cần xử lý cả hai kết quả.

Điểm danh bị từ chối (sai vị trí, ngoài giờ, ...) thì **chưa tính là đã điểm danh**, sinh viên thử lại được.

5. Menu drawer

`GET /api/menu`

Trả về danh sách menu của drawer nav, đã sắp xếp theo `ordering` tăng dần. Tên trường dùng snake_case, khớp với `MenuNavItem.fromMap` trong app.

`200`

```
[
  { "id": 1, "icon_name": "newspaper", "menu_name": "Tin tức UEH", "menu_url":
"https://ueh.edu.vn/tintucmobile", "ordering": 0 },
  { "id": 2, "icon_name": "calendarDays", "menu_name": "Sự kiện UEH", "menu_url":
"", "ordering": 1 }
]
```

Trường	Ý nghĩa
`icon_name`	Tên icon FontAwesome để app ánh xạ sang icon (`newspaper`, `calendarDays`, `chartLine`, ...)
`menu_url`	Liên kết của mục menu, chuỗi rỗng nếu chưa có

Mã	Khi nào
`401`	Thiếu token, token sai hoặc hết hạn (body rỗng)

6. Tin tức

Ba API tin tức **không cần token**. Danh sách không kèm nội dung đầy đủ (`content`) để response nhẹ; nội dung chỉ có ở API chi tiết.

Mỗi tin trong danh sách có dạng:

```
{
  "uid": 12,
  "cat_uid": 219,
  "category_name": "Tin UEH",
  "title": "Tiêu đề tin",
  "short_desc": "Mô tả ngắn",
  "post_date": "2026-09-01T00:00:00",
  "sdgs": [4, 13],
  "is_featured": true,
  "background_img":
"https://api.ngoisaoviet.info/images/upload/img_background/ueh-background-
639274789085988285.png",
  "thumbnail_img": "https://api.ngoisaoviet.info/images/upload/img_thumbnail/ueh-
thumbnail-639274789085988285.png"
}
```

`background_img` và `thumbnail_img` là URL đầy đủ, dùng trực tiếp được. Cả hai có thể là `null` (tin không có ảnh nền hoặc ảnh thu nhỏ), app cần xử lý từng trường riêng.

`sdgs` là danh sách Uid SDG (từ 1 đến 17) của tin, **từ 1 đến 3 phần tử** theo thứ tự SDG1, SDG2, SDG3. Các SDG không trùng nhau. Mỗi số tương ứng một icon SDG của app:

Số	Mục tiêu
1	Xóa nghèo

Số	Mục tiêu
2	Không còn nạn đói
3	Sức khỏe và cuộc sống tốt
4	Giáo dục có chất lượng
5	Bình đẳng giới
6	Nước sạch và vệ sinh
7	Năng lượng sạch với giá thành hợp lý
8	Công việc tốt và tăng trưởng kinh tế
9	Công nghiệp, sáng tạo và phát triển hạ tầng
10	Giảm bất bình đẳng
11	Các thành phố và cộng đồng bền vững
12	Tiêu thụ và sản xuất có trách nhiệm
13	Hành động về khí hậu
14	Tài nguyên và môi trường biển
15	Tài nguyên và môi trường trên đất liền
16	Hòa bình, công lý và các thể chế mạnh mẽ
17	Quan hệ đối tác vì các mục tiêu

6.1. Tin nổi bật

`GET /api/news/featured`

Trả về **tối đa 5 tin**, theo thứ tự:

1. Một tin nổi bật (`is_featured = true`) mới nhất.
2. Tin mới nhất của mục `219`, `220`, `221`, `222` (lần lượt).

Tin nổi bật luôn khác 4 tin còn lại: nếu tin nổi bật mới nhất cũng là tin mới nhất của một trong 4 mục thì hệ thống lấy tin nổi bật kế tiếp. Mục nào chưa có tin, hoặc không còn tin nổi bật phù hợp, thì bỏ qua nên mảng có thể ít hơn 5 phần tử.

`200`: mảng các tin như trên.

6.2. Danh sách tin theo mục

`GET /api/news?cat_uid=219&limit=20&offset=0`

Tham số	Bắt buộc	Mô tả
`cat_uid`	Có	Uid của mục

Tham số	Bắt buộc	Mô tả
`limit`	Không	Số tin mỗi lần, từ 1 đến 100, mặc định 20
`offset`	Không	Số tin bỏ qua, từ 0, mặc định 0

Tin sắp xếp theo ngày đăng **mới nhất trước**.

`200`

```
{
  "total": 125,
  "limit": 20,
  "offset": 0,
  "items": [ { "uid": 12, "title": "..." } ]
}
```

`total` là tổng số tin của mục (không phụ thuộc `limit`, `offset`). Còn tin để tải thêm khi `offset + items.length < total`. Mục không có tin hoặc `offset` quá lớn thì `items` là mảng rỗng.

Mã	Khi nào
`400`	Thiếu `cat_uid`, `limit` ngoài 1-100 hoặc `offset` âm: `{ "message": "..." }`

6.3. Chi tiết tin

`GET /api/news/{uid}`

`200`: như một tin ở trên, thêm trường `content` là **HTML** của bài viết (có thể rất dài, hàng trăm KB). App nên hiển thị bằng WebView hoặc widget render HTML và chỉ tải khi mở bài. Nếu dùng reverse proxy (Nginx, IIS) thì nên bật gzip cho JSON để giảm dung lượng.

Mã	Khi nào
`404`	Không có tin với `uid` đó: `{ "message": "Không tìm thấy tin tức." }`

Lỗi 400: thiếu hoặc sai dữ liệu

Body thiếu trường bắt buộc hoặc JSON sai định dạng sẽ nhận `400` theo chuẩn ASP.NET Core:

```
{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": { "MSSV": ["The MSSV field is required."] }
}
```

Lỗi nghiệp vụ (`401`, `403`, `404`, `409` và các lỗi `400` ở mục điểm danh và tin tức) thì đọc trường `message`.

Ví dụ JavaScript

Tự làm mới token khi gặp `401` và chỉ thử lại một lần.

```
const BASE = "https://api.ngoisaoviet.info";
let tokens = null; // { token, refresh_token }

async function post(path, body, token) {
  return fetch(`${BASE}${path}`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      ...(token && { Authorization: `Bearer ${token}` }),
    },
    body: JSON.stringify(body),
  });
}

async function login(mssv, matKhau) {
  const res = await post("/api/login", { mssv, mat_khau: matKhau });
  if (!res.ok) throw new Error((await res.json()).message);
  tokens = await res.json();
}

async function refresh() {
  const res = await post("/api/refresh-token", { refresh_token:
tokens.refresh_token });
  if (!res.ok) { tokens = null; return false; } // phải đăng nhập lại
  tokens = await res.json();                    // lưu ngay cặp token mới
  return true;
}

async function callApi(method, path, body, retry = true) {
  const res = await fetch(`${BASE}${path}`, {
    method,
    headers: { "Content-Type": "application/json", Authorization: `Bearer
${tokens.token}` },
    body: body && JSON.stringify(body),
  });
  if (res.status === 401 && retry && (await refresh())) return callApi(method,
path, body, false);
  return res;
}

await login("31240000000", "********");
const info = await (await callApi("GET", "/api/student-info")).json();
const result = await callApi("POST", "/api/attendance", {
  code: "A7K2P9", latitude: 10.7610401, longitude: 106.6684314,
});
console.log(result.status, await result.json());

// Menu drawer (cần token)
```

```
const menu = await (await callApi("GET", "/api/menu")).json();

// Tin tức: không cần token, dùng fetch thường
const featured = await (await fetch(`${BASE}/api/news/featured`)).json();

// Phân trang tin theo mục: tải đến khi hết
const items = [];
let offset = 0;
while (true) {
  const page = await (await fetch(`${BASE}/api/news?
cat_uid=219&limit=20&offset=${offset}`)).json();
  items.push(...page.items);
  offset += page.items.length;
  if (page.items.length === 0 || offset >= page.total) break;
}

// Chi tiết một tin (content là HTML)
const detail = await (await fetch(`${BASE}/api/news/${items[0].uid}`)).json();
```

Lưu ý bảo mật

- Lưu `token` và `refresh_token` ở nơi an toàn (Keychain trên iOS, Keystore trên Android), không ghi log.
- Không nhúng mật khẩu vào app, chỉ hỏi sinh viên khi đăng nhập.
- Tọa độ do thiết bị gửi lên, server chỉ kiểm tra được khoảng cách. App nên lấy tọa độ trực tiếp từ dịch vụ định vị của hệ điều hành, không cho người dùng tự nhập.